RecoverAI: Predictive Deleted File Recovery

CARP: Confidence-Adaptive Recovery Prioritization

Major Project External Viva Comprehensive Preparation Guide

Candidate: Polusani Shashwath
Role: Dependency & Recovery Surface
Email: polusanishashwath123@gmail.com
Affiliation: VIT-AP University (SCOPE)

Chapter 1: Role Overview & Executive Briefing

Role Definition: As the **Dependency & Recovery Surface Specialist** and Systems Architect, my core responsibility is system reliability, dependency containerization, forensic surface minimization, data persistence, and rigorous automated testing across the entire RecoverAI pipeline. I ensure that forensic operations never contaminate evidence media, that all C and Python dependencies run deterministically across platforms, and that our automated test suites mathematically verify system constraints.

Spoken Pitch for External Examiners (30-Second Version):

"Good morning, respected examiners. My name is Polusani Shashwath, and my individual responsibility is Dependency Management and Recovery Surface Architecture. In digital forensics, tools must guarantee absolute environment reproducibility, strict chain of custody, and zero evidence contamination. I architected our containerized Docker deployment, managed Linux and C-extension dependencies for xCarver's Aho-Corasick scanner, engineered our SQLAlchemy SQLite relational database layer across six forensic tables, and authored our automated pytest suite that formally verifies budget caps and cost-value monotonicity across our recovery tiers."

The 1-Minute Technical Summary:

"Digital forensics software operates in high-stakes environments where courtroom admissibility depends on immutable auditability and zero host contamination. In my role, I mapped and minimized the recovery attack surface: establishing read-only disk block access (open in rb mode) to prevent evidence tampering, and containing volatile memory execution to prevent malicious code embedded in carved polyglots from executing during inspection. To eliminate fragmented CSV logging, I architected db_layer.py using SQLAlchemy 2.0 ORM over an ACID-compliant SQLite database across six normalized relational tables, permanently binding experiment runs to SHA-256 ground truth hashes and knapsack allocations. Finally, I authored our four automated test suites in tests/, writing property-based test cases that mathematically prove cost-yield monotonicity and budget ceiling enforcement."

Chapter 2: Technical Deep Dive & Systems Architecture

1. Forensic Recovery Surface & Chain of Custody Minimization:

The recovery surface defines the physical and logical boundaries where RecoverAI interacts with evidence drives, memory buffers, and operating system kernels:

- **Read-Only Block Stream Isolation:** Disk images and raw sector buffers are accessed strictly in read-only binary mode (open(path, 'rb') or losetup -r). No temporary files, swap space, or indexing caches are ever written back to the source media.
- **Active Memory Containment:** Carved payload extraction and vector embedding generation occur in isolated user-space memory buffers, neutralizing weaponized polyglots or corrupted binaries embedded in unallocated sectors.
- **Air-Gapped Operational Independence:** The entire stack (PyTorch, ChromaDB, Sentence-Transformers, SQLite) runs 100% locally on CPU/GPU without outbound internet dependencies, preserving evidentiary confidentiality.

2. Containerized Dependency Management (Dockerfile & docker-compose.yml):

Forensic reproducibility requires identical runtime environments. I built a containerized Ubuntu 22.04 LTS deployment packaging:
• **System C Toolchain:** gcc, make, and libc-dev required to compile xCarver's multithreaded Aho-Corasick C scanner (scanner.c).
• **Python Runtime:** Python 3.11 with pinned wheels for PyTorch 2.2+, XGBoost 2.0, Sentence-Transformers, ChromaDB, and SQLAlchemy 2.0.
• **WSL2 Kernel Boundary Handling:** Standardized environment variables and loop device permissions bridging Windows 11 host storage to Linux virtual filesystems.

3. Relational Persistence Layer (db_layer.py):

I authored db_layer.py, replacing loose flat CSV exports with a structured SQLite database via SQLAlchemy 2.0 ORM across six normalized relational tables:

• experiments: Tracks unique run_id, timestamps, git commit hash, and condition counts.
• manifest: Stores pre-deletion metadata (original filename, size_bytes, pre-deletion SHA-256).
• verdicts: Stores post-carving verification (recovered SHA-256, verdict enum, carving execution time).
• predictions: Logs out-of-fold ML probabilities (p_full, p_partial, expected_fraction).
• allocations: Logs knapsack scheduler assignments (effort_level, cost_allocated_s, yield).
• search_index: Maps carved files to ChromaDB document IDs and embedding model versions.

4. Automated Testing & Verification Suite (tests/):

I authored and maintained four automated test modules in tests/:

- test_orchestrator.py: Formally proves cost-value monotonicity: cost(SKIP) < cost(HEADER_CHECK) < cost(SCOPED_CARVE) < cost(FULL_CARVE) and proves the knapsack optimizer never exceeds budget B.
- test_predictor.py: Verifies 7-dimensional feature vectors and confirms softmax probabilities sum to 1.0.
- test_semantic.py: Validates 384-dimensional embedding norms and verifies fused score monotonicity.
- test_integration.py: End-to-end integration smoke test verifying the complete recovery lifecycle.

Table Name	Primary Keys & FKs	Forensic Purpose & Relational Link
experiments	id (PK), run_id (Unique)	Audit trail recording run metadata, timestamps, and commit hashes
manifest	id (PK), experiment_id (FK)	Pre-deletion baseline storing original file names, sizes, and SHA-256
verdicts	id (PK), experiment_id (FK)	Physical ground truth storing recovered SHA-256 and match ratios
predictions	id (PK), experiment_id (FK)	ML audit log tracking predicted probabilities and survival fractions
allocations	id (PK), experiment_id (FK)	Scheduler audit log tracking effort tiers, costs, and expected yields
search_index	id (PK), experiment_id (FK)	ChromaDB linkage cross-referencing vector IDs with database records

Chapter 3: Codebase Ownership & Key Functions

- db_layer.py: Implements SQLAlchemy ORM models, database engine initialization, and transaction session makers.
- Dockerfile & docker-compose.yml: Configures multi-stage containerized execution, toolchain setup, and volumes.
- common/schema.py: Defines immutable dataclasses and enums (Filesystem, Medium, EffortLevel).
- tests/test_orchestrator.py: Implements test_cost_value_monotonicity() and test_budget_knapsack_allocation().
- tests/test_integration.py: Implements test_full_recoverai_pipeline() smoke test.

Chapter 4: Cross-Module Integration (How My Work Connects)

1. **Integration with Abdul (ML & Knapsack):** I test Abdul's knapsack allocation logic in test_orchestrator.py and persist his model predictions and knapsack schedules into the predictions and allocations database tables.
2. **Integration with Asmin (Threat Model & Raw Carver):** I manage the C-compiler toolchain and loop device mount permissions needed for Asmin's xCarver scanner, and persist her SHA-256 ground truth verdicts into the verdicts table.
3. **Integration with Chandrika (Semantic Search):** I test Chandrika's embedding dimensions and fused scoring logic in test_semantic.py, and link her ChromaDB vector collection to the search_index table.

Chapter 5: Top 25 Viva Questions & Defense Answers for Shashwath

Q1: What was your individual role in RecoverAI?

I was the Dependency & Recovery Surface Specialist. I architected our Docker containerization, managed C and Python dependencies, minimized the forensic attack surface, built the SQLite relational database layer, and authored the automated pytest verification suite.

Q2: Why use an embedded SQLite database instead of PostgreSQL or MySQL?

Digital forensics workstations operate in air-gapped, isolated environments. A client-server database like PostgreSQL introduces external server daemons, port vulnerabilities, and network configuration overhead. SQLite is an ACID-compliant, self-contained database stored as a single file that can be archived directly with case evidence.

Q3: How do you guarantee zero evidence contamination on disk images?

Disk images are accessed strictly in read-only binary mode using 'rb' flags or read-only loop devices ('losetup -r'). No temporary files or indexing data are ever written to the evidence media.

Q4: What is the recovery surface and how did you minimize it?

The recovery surface is the logical boundary where the tool touches raw disk sectors, memory buffers, and OS APIs. I minimized it by restricting file operations to read-only block streams and isolating carved payload inspection in memory to prevent polyglot exploit execution.

Q5: What are the 6 tables in your database schema?

experiments, manifest, verdicts, predictions, allocations, and search_index.

Q6: How does test_cost_value_monotonicity verify strategy logic?

It tests that time costs and accessible yields increase strictly monotonically across effort tiers: cost(SKIP) < cost(HEADER_CHECK) < cost(SCOPED_CARVE) < cost(FULL_CARVE). This proves the strategy model is mathematically sound.

Q7: What does test_budget_knapsack_allocation test?

It tests that when budget B = 0.0s, all files are assigned SKIP. It also tests that under budget B = 0.50s, the sum of allocated costs never exceeds 0.50s.

Q8: Why is Docker containerization important for forensic software?

Courtroom admissibility requires complete reproducibility. A container encapsulates the exact compiler (gcc), C-libraries, Python version (3.11), and ML framework versions, guaranteeing identical behavior regardless of the host OS.

Q9: What C dependencies were required for xCarver v4?

gcc, make, and libc-dev were required to compile scanner.c, which implements the multithreaded Aho-Corasick automaton.

Q10: What happens if an image path does not exist when db_layer writes records?

SQLAlchemy transactions wrap inserts in try-except blocks with session.rollback(), preventing partial database writes and maintaining database integrity.

Q11: How do you handle schema migrations or versioning?

Our schema is managed declaratively via SQLAlchemy declarative_base(), allowing automated table creation on initial startup via Base.metadata.create_all(engine).

Q12: What is the purpose of the allocations table?

It records the knapsack scheduler's operational decision for every file, logging the assigned tier (SKIP, FULL_CARVE), allocated cost, and expected yield for forensic auditing.

Q13: How does test_integration.py validate the full pipeline?

It executes an end-to-end smoke test: creating a synthetic disk image -> predicting recoverability -> running knapsack carving -> indexing in ChromaDB -> querying via natural language.

Q14: How did you test ChromaDB embedding dimensionality?

In test_semantic.py, test_embedding_dimensions asserts that embed_text() returns a vector of length exactly 384 and has an L2 norm of 1.0 +/- 10^-4.

Q15: How does your database ensure chain of custody?

Every experiment run generates a unique run_id timestamped at start and finish, tied to the git commit hash and storing SHA-256 pre- and post-carving hashes.

Q16: Why did you use SQLAlchemy ORM instead of raw sqlite3 queries?

SQLAlchemy provides strict type safety, relationship mapping between foreign keys (e.g., verdicts tied to experiments), and connection pooling while preventing SQL injection vulnerabilities.

Q17: What environment variable configures your database path?

DB_PATH, which defaults to './db/recoverai.db' if not explicitly defined.

Q18: How does the system handle corrupted or zero-byte files during text extraction?

extract_text() catches extraction exceptions and falls back to filename string embeddings, preventing crashes on corrupted documents.

Q19: What is the primary bottleneck in disk I/O during testing?

Repeatedly mounting and unmounting ext4 loop devices in WSL2. We mitigated this by testing against in-memory raw binary streams where possible.

Q20: How do you verify that Softmax probabilities sum to 1.0?

In test_predictor.py, we assert that torch.sum(probs, dim=1) equals 1.0 within a floating-point tolerance of 10^-5.

Q21: How does your module connect to Abdul's work?

I test Abdul's knapsack solver in test_orchestrator.py and persist his model predictions and allocations in the database.

Q22: How does your module connect to Asmin's work?

I manage the C-compiler dependencies and loop device mount permissions for Asmin's xCarver engine, and persist her ground truth verdicts.

Q23: How does your module connect to Chandrika's work?

I validate Chandrika's vector dimensions and search scoring in test_semantic.py and maintain the search_index table mapping vector IDs.

Q24: What happens if an investigator provides a corrupted SQLite file?

The database engine initializes with WAL (Write-Ahead Logging) mode, enabling automatic crash recovery and transaction rollback.

Q25: What is the single most important architectural improvement you would introduce?

Migrating raw sector reads to asynchronous direct I/O (O_DIRECT / io_uring in Linux) to bypass OS buffer cache and achieve true non-blocking disk saturation.

Chapter 6: Rapid Revision Cheat Sheet for Shashwath

- ***Database Schema:*** *6 relational tables: experiments, manifest, verdicts, predictions, allocations, search_index*
- ***ORM Framework:*** *SQLAlchemy 2.0 over self-contained, air-gapped SQLite database (db_layer.py)*
- ***Forensic Integrity:*** *Read-only block access ('rb'), immutable SHA-256 auditing, zero evidence contamination*
- ***Testing Suites:*** *4 pytest modules in tests/: test_orchestrator.py, test_predictor.py, test_semantic.py, test_integration.py*
- ***Monotonicity Rule:*** *cost(SKIP) < cost(HDR) < cost(SCOPED) < cost(FULL) mathematically verified*
- ***Containerization:*** *Ubuntu 22.04 LTS Docker container with Python 3.11, PyTorch 2.2+, and C compiler (gcc, make)*